

# Message Broker + Django Redis Setup on Windows (WSL)

---

## Part 1 -- Message Broker

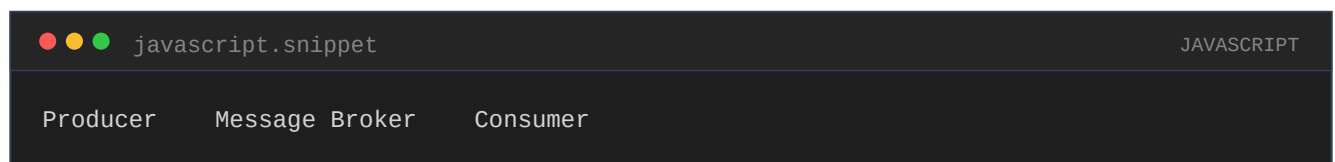
### 1. What is a Message Broker?

A **Message Broker** is middleware that helps applications or services **send, receive, store, and deliver messages** between each other.

Simply:

*A Message Broker is a message delivery system between applications or services.*

It acts as an intermediate layer between:



The producer and consumer do not necessarily need to communicate directly with each other.

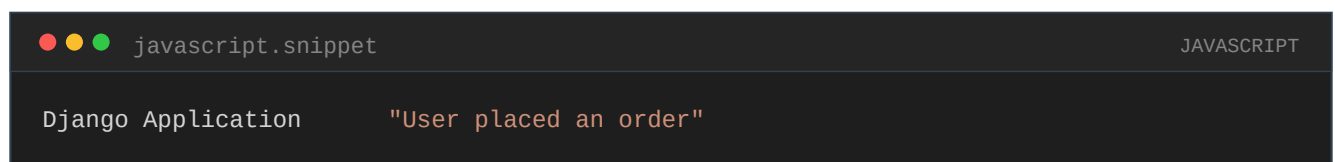
---

## 2. Producer and Consumer

### Producer

The **Producer** creates and sends a message.

Example:



The Django application is the producer.

---

### Consumer

The **Consumer** receives and processes the message.

Example:

```
javascript.snippet JAVASCRIPT
"User placed an order"      Payment Worker
```

The payment worker is the consumer.

---

### 3. What Does a Message Broker Do?

A message broker commonly performs several tasks.

#### 1. Receive Messages

The broker receives messages from producers.

```
javascript.snippet JAVASCRIPT
Producer    Broker
```

---

#### 2. Store/Queue Messages

If a consumer is temporarily unavailable or busy, messages can be retained according to the broker's delivery model.

```
javascript.snippet JAVASCRIPT
Producer    Broker    Queue    Consumer
```

---

#### 3. Deliver Messages

The broker delivers messages to the appropriate consumer or consumers according to the messaging pattern.

---

#### 4. Reliability

Depending on the broker and configuration, it can provide features such as:

```
javascript.snippet JAVASCRIPT

AcknowledgementsRetriesPersistenceDead-letter handlingMessage orderingDelivery
guarantees
```

These features are **not identical across all message brokers**.

## 4. Why Do We Use a Message Broker?

### 4.1 Decoupling

Without a broker:

```
javascript.snippet JAVASCRIPT

Django Email Service
```

The Django application directly depends on the email service.

With a broker:

```
javascript.snippet JAVASCRIPT

Django Message Broker Email Worker Email Service
```

The producer and consumer are more loosely coupled.

## 5. Asynchronous Processing

One of the most common reasons for using a message broker is **asynchronous processing**.

Example:

```
javascript.snippet JAVASCRIPT

User Signup Django "Send Welcome Email" Broker Email Worker
```

The Django request does not need to wait for the email to actually be sent.

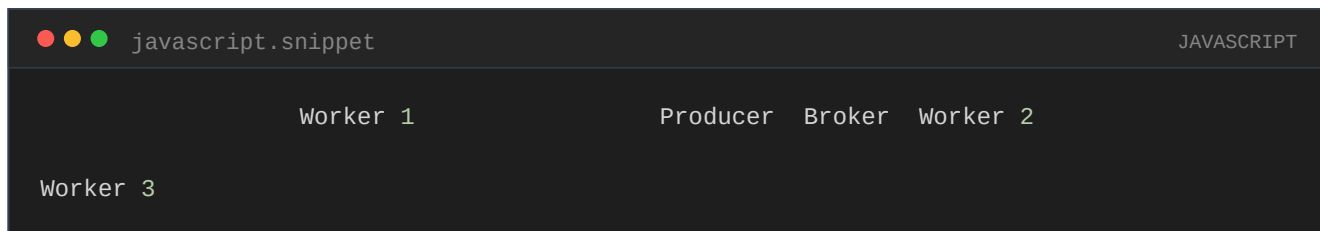
Therefore, a slow email operation does not necessarily block the user's request.

---

## 6. Scalability

Suppose one worker cannot process all messages.

You can add more workers:



This allows the system to process more work concurrently, depending on the workload and broker/consumer configuration.

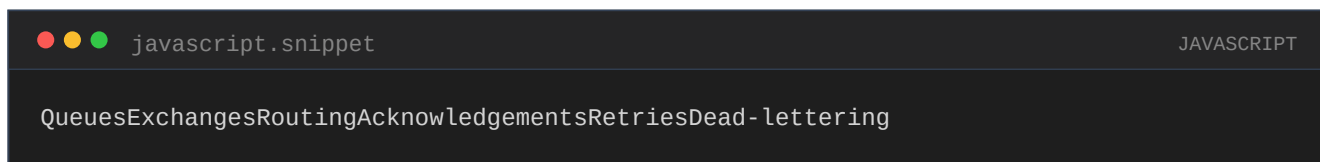
---

## 7. Common Message Brokers and Messaging Systems

### RabbitMQ

RabbitMQ is a general-purpose message broker.

Common features include:

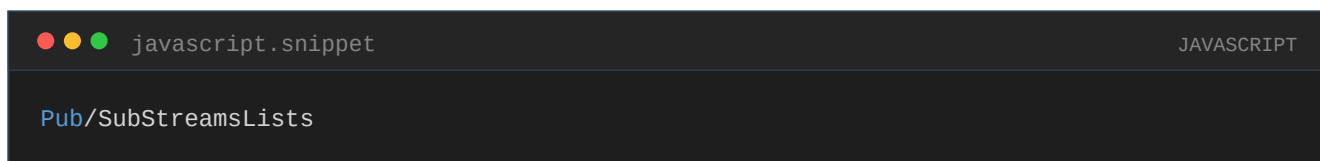


It is widely used for task queues and asynchronous application communication.

---

### Redis

Redis is primarily an **in-memory data store**, but it also provides messaging features such as:



Redis can also be used as a broker/backend for systems such as Celery.

Important:

*Redis itself is not simply synonymous with "message broker"; its role depends on which Redis feature and application architecture you are using.*

---

## Apache Kafka

Kafka is a **distributed event streaming platform** designed for high-throughput, durable event streams.

It is commonly used for:

```
javascript.snippet JAVASCRIPT
Event streamingLog pipelinesData integrationReal-time analyticsLarge-scale event processing
```

Kafka's architecture and semantics are different from a traditional queue-oriented broker such as RabbitMQ.

---

## 8. Message Broker vs Task Queue

These concepts are related but not identical.

### Message Broker

Responsible for transporting messages between systems.

```
javascript.snippet JAVASCRIPT
Producer Broker Consumer
```

### Task Queue

Used to distribute background jobs/tasks to workers.

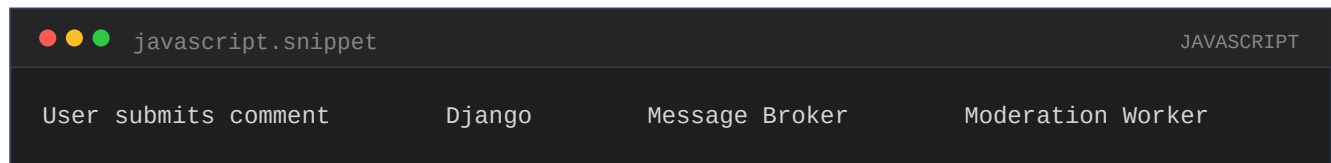
```
javascript.snippet JAVASCRIPT
Application Task Queue Worker
```

For example, **Celery** is a distributed task queue system that commonly uses a broker such as Redis or RabbitMQ.

---

## 9. Real-Life Examples

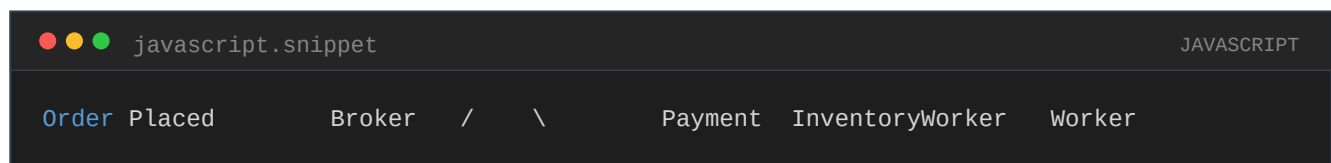
### Example 1 -- Web Application



The moderation task can happen in the background.

---

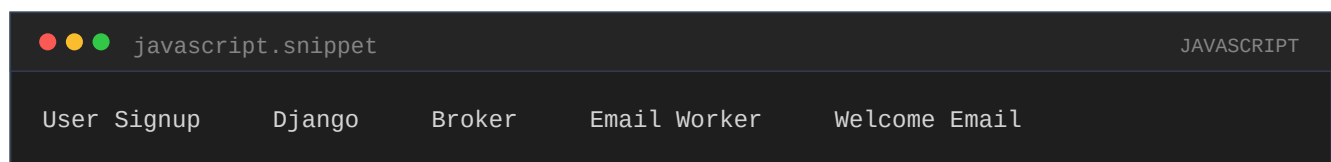
### Example 2 -- E-commerce



Different consumers can process different parts of the workflow.

---

### Example 3 -- Email

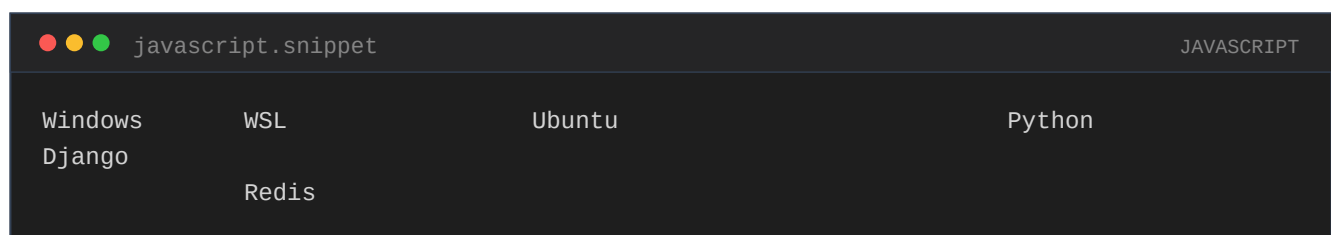


---

## Part 2 -- Django + Redis Setup on Windows Using WSL

### Architecture

Our development environment will look like:



Django can then use Redis for purposes such as:

```
javascript.snippet JAVASCRIPT

CachingSessionsCelery brokerCelery result backendDjango Channels channel layer
```

These are separate use cases even though they can use the same Redis server.

---

## Step 1 -- Install WSL

Open **PowerShell as Administrator**.

Run:

```
javascript.snippet JAVASCRIPT

wsl --install
```

This installs WSL and, on supported Windows versions, a default Linux distribution such as Ubuntu.

Restart Windows if requested.

Then open Ubuntu from the Start menu.

---

## Step 2 -- Check WSL

Inside Ubuntu:

```
javascript.snippet JAVASCRIPT

wsl --status
```

You can also check the Linux distribution/version:

```
javascript.snippet JAVASCRIPT

lsb_release -a
```

### Important

The command:

```
javascript.snippet JAVASCRIPT
wsl --install
```

does not specifically mean "install WSL 1."

Modern WSL installations generally use **WSL 2**, which is the recommended architecture for most development use cases.

---

## Step 3 -- Update Ubuntu

Run:

```
javascript.snippet JAVASCRIPT
sudo apt update
```

Then:

```
javascript.snippet JAVASCRIPT
sudo apt upgrade -y
```

---

## Step 4 -- Install Redis

Install Redis:

```
javascript.snippet JAVASCRIPT
sudo apt install redis-server -y
```

Depending on your Ubuntu/WSL setup, start Redis with:

```
javascript.snippet JAVASCRIPT
sudo service redis-server start
```

Check Redis:

```
javascript.snippet JAVASCRIPT
redis-cli ping
```

Expected output:

```
javascript.snippet JAVASCRIPT
PONG
```

---

## Step 5 -- Check Redis Status

You can also try:

```
javascript.snippet JAVASCRIPT
redis-cli
```

Then:

```
javascript.snippet JAVASCRIPT
PING
```

Expected:

```
javascript.snippet JAVASCRIPT
PONG
```

Exit:

```
javascript.snippet JAVASCRIPT
exit
```

---

## Step 6 -- Check Python

Check Python:

```
javascript.snippet JAVASCRIPT

python3 --version
```

Check pip:

```
javascript.snippet JAVASCRIPT

pip3 --version
```

For a Django project, using a virtual environment is recommended.

Install venv support if necessary:

```
javascript.snippet JAVASCRIPT

sudo apt install python3-venv -y
```

---

## Step 7 -- Create a Virtual Environment

Create:

```
javascript.snippet JAVASCRIPT

python3 -m venv .venv
```

Activate:

```
javascript.snippet JAVASCRIPT

source .venv/bin/activate
```

After activation, your terminal may show:

```
javascript.snippet JAVASCRIPT

(.venv)
```

This keeps project dependencies isolated.

---

## Step 8 -- Install Django

---

Inside the virtual environment:

```
javascript.snippet JAVASCRIPT  
  
python -m pip install django
```

Check:

```
javascript.snippet JAVASCRIPT  
  
django-admin --version
```

---

## Step 9 -- Create Django Project

Create the project:

```
javascript.snippet JAVASCRIPT  
  
django-admin startproject myproject
```

Move into the project:

```
javascript.snippet JAVASCRIPT  
  
cd myproject
```

Create an app:

```
javascript.snippet JAVASCRIPT  
  
python manage.py startapp myapp
```

Project structure:

```
javascript.snippet JAVASCRIPT  
  
myproject/ manage.py myproject/  __init__.py  settings.py  urls.py  asgi.py  
wsgi  
.py myapp/  migrations/  __init__.py  admin.py  apps.py  models.py  
tests  
.py  views.py
```

---

## Step 10 -- Install Redis-related Python Packages

Install the packages you actually need.

For Django Redis cache:

JAVASCRIPT

```
python -m pip install redis django-redis
```

For Celery:

JAVASCRIPT

```
python -m pip install celery
```

For Django Channels:

JAVASCRIPT

```
python -m pip install channels channels-redis
```

You can install all together:

JAVASCRIPT

```
python -m pip install redis django-redis celery channels channels-redis
```

---

## Part 3 -- Configure Django Cache with Redis

### Step 11 -- settings.py

Add:

```
javascript.snippet JAVASCRIPT

CACHES = {      "default": {      "BACKEND": "django_redis.cache.RedisCache",
"LOCATION"
: "redis://127.0.0.1:6379/1",      "OPTIONS": {      "CLIENT_CLASS":
"django_redis.client.DefaultClient"
,      },    }}
```

Here:

```
javascript.snippet JAVASCRIPT

127.0.0.1 Redis server address6379 Redis default port/1 Redis database number 1
```

Redis supports multiple logical databases identified by numbers, although separate Redis instances or key namespaces are often preferable for stronger isolation in production architectures.

---

## Part 4 -- Store Django Sessions in Redis

You can configure Django sessions to use the cache:

```
javascript.snippet JAVASCRIPT

SESSION_ENGINE = (      "django.contrib.sessions.backends.cache")SESSION_CACHE_ALIAS =
"default"
```

This tells Django to store sessions through the configured cache backend.

### Important

With cache-based sessions, session data depends on the availability of the cache. For production systems, choose the session strategy according to your persistence and availability requirements.

---

## Part 5 -- Test Redis Cache

### Step 12 -- Create a View

In `views.py`:

```
javascript.snippet JAVASCRIPT

from django.core.cache import cache
from django.http import HttpResponseRedirect

def home(request):
    value = cache.get("visits", 0)
    value += 1
    cache.set("visits", value, timeout=60)
    return HttpResponseRedirect(f"This page was visited {value} times.")
```

---

## Step 13 -- Configure URL

In `urls.py`:

```
javascript.snippet JAVASCRIPT

from django.contrib import admin
from django.urls import path
from myapp.views import home
urlpatterns = [
    path("admin/", admin.site.urls),
    path("", home),
]
```

---

## Step 14 -- Test the Cache

Run:

```
javascript.snippet JAVASCRIPT

python manage.py runserver
```

Open:

```
javascript.snippet JAVASCRIPT

http://127.0.0.1:8000/
```

You may see:

```
javascript.snippet JAVASCRIPT

This page was visited 1 times.
```

Refresh:

```
javascript.snippet JAVASCRIPT

This page was visited 2 times.
```

The value is stored in Redis for 60 seconds.

After the cache expires, it starts again from:

```
javascript.snippet JAVASCRIPT

1
```

---

## Part 6 -- Celery + Redis

### Step 15 -- What is Celery?

**Celery** is a distributed task queue used to execute tasks asynchronously.

Architecture:

```
javascript.snippet JAVASCRIPT

Django    Celery Task    Redis Broker    Celery Worker
```

Example:

```
javascript.snippet JAVASCRIPT

User Signup    Django    Celery Task    Redis    Worker    Send Email
```

---

### Step 16 -- Create celery.py

Inside the Django project package:

```
javascript.snippet JAVASCRIPT

myproject/    celery.py
```

Add:

```
javascript.snippet JAVASCRIPT

import os
from celery import Celery

os.environ.setdefault("DJANGO_SETTINGS_MODULE",
    "myproject.settings")
app = Celery("myproject").config_from_object("django.conf:
    settings", namespace="CELERY")
app.autodiscover_tasks()
```

---

## Step 17 -- Configure Celery in settings.py

Use Redis as the broker:

```
javascript.snippet JAVASCRIPT

CELERY_BROKER_URL = "redis://127.0.0.1:6379/0"
```

If you want to store Celery task results in Redis:

```
javascript.snippet JAVASCRIPT

CELERY_RESULT_BACKEND = "redis://127.0.0.1:6379/0"
```

Here we are using:

```
javascript.snippet JAVASCRIPT

Redis DB 0 Celery Redis DB 1 Django cache
```

This separation helps keep the use cases organized.

---

## Step 18 -- Create a Celery Task

In:

```
javascript.snippet JAVASCRIPT

myapp/tasks.py
```

Add:

```
from celery import shared_task@shared_taskdef add(x, y):    return x + y
```

---

## Step 19 -- Run Celery Worker

Keep Redis running.

Open another WSL terminal.

Activate the virtual environment:

```
source .venv/bin/activate
```

Go to the project directory and run:

```
celery -A myproject worker -l info
```

The worker should start and discover the task.

---

## Step 20 -- Test Celery

From Django shell:

```
python manage.py shell
```

Then:

```
from myapp.tasks import addresult = add.delay(10, 20)
```

`delay()` sends the task asynchronously.

You can inspect the result:

```
javascript.snippet JAVASCRIPT

result.get()
```

Expected:

```
javascript.snippet JAVASCRIPT

30
```

For production applications, avoid blocking the web request by waiting for asynchronous results unless that behavior is specifically required.

---

## Part 7 -- Django Channels + Redis

### Step 21 -- What is Django Channels?

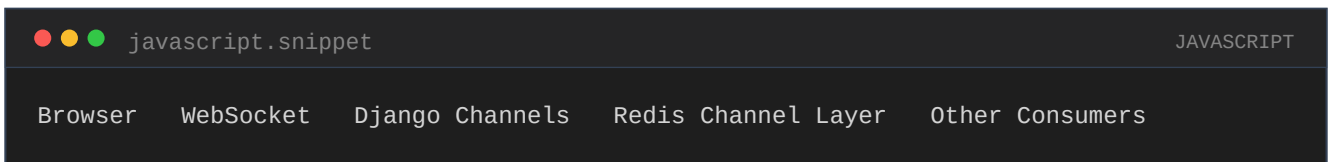
Django Channels extends Django beyond traditional HTTP request/response handling.

It is commonly used for:

```
javascript.snippet JAVASCRIPT

WebSocketsReal-time communicationChat applicationsLive notificationsReal-time updates
```

Architecture:



---

### Step 22 -- Configure Channels

In settings.py:

```
javascript.snippet JAVASCRIPT

CHANNEL_LAYERS = {      "default": {      "BACKEND": (
"channels_redis.core.RedisChannelLayer"
),      "CONFIG": {      "hosts": [      ("127.0.0.1", 6379)

],      },      },}
```

---

## Step 23 -- Important: Channels Requires ASGI

For Channels, Django's ASGI configuration needs to be set up appropriately.

In `settings.py`:

```
javascript.snippet JAVASCRIPT

ASGI_APPLICATION = "myproject.asgi.application"
```

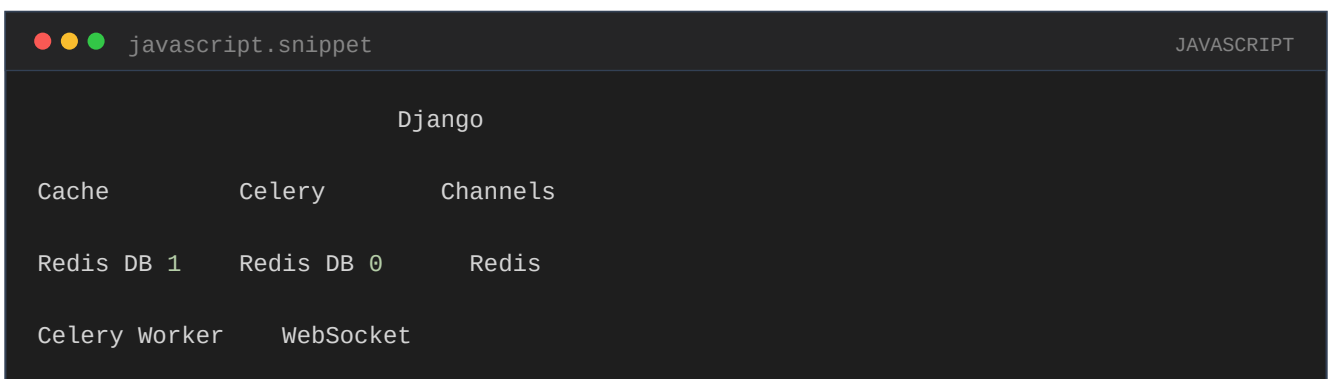
The project also needs an appropriate `asgi.py` configuration and WebSocket routing/consumer setup.

Simply installing `channels` and configuring `CHANNEL_LAYERS` does not by itself create a working WebSocket application.

---

## Part 8 -- Final Project Architecture

After configuring everything, the architecture can look like:



A single Redis server can support multiple use cases, but production deployments should consider isolation, persistence, memory limits, security, and failure scenarios.

---

## Part 9 -- Run the Project

### Step 24 -- Apply Migrations

 javascript.snippet

JAVASCRIPT

```
python manage.py migrate
```

---

### Step 25 -- Start Redis

Depending on your WSL/Ubuntu service setup:

 javascript.snippet

JAVASCRIPT

```
sudo service redis-server start
```

Test:

 javascript.snippet

JAVASCRIPT

```
redis-cli ping
```

Expected:

 javascript.snippet

JAVASCRIPT

```
PONG
```

---

### Step 26 -- Start Django

 javascript.snippet

JAVASCRIPT

```
python manage.py runserver
```

Open:

```
javascript.snippet JAVASCRIPT
http://127.0.0.1:8000/
```

---

## Step 27 -- Start Celery

Open another terminal:

```
javascript.snippet JAVASCRIPT
source .venv/bin/activate
```

Then:

```
javascript.snippet JAVASCRIPT
celery -A myproject worker -l info
```

Now the basic architecture is:



---

## Part 10 -- Redis Database Numbers

In our example:

```
javascript.snippet JAVASCRIPT
Redis DB 0  Celery DB 1  Django Cache
```

URLs:

```
javascript.snippet JAVASCRIPT

CELERY_BROKER_URL = ("redis://127.0.0.1:6379/0")
CACHES = { "default": {
    "BACKEND":
    : "django_redis.cache.RedisCache",
    "LOCATION": "redis://127.0.0.1:6379/1",
    }}

```

The /0 and /1 represent Redis logical database numbers.

---

## Part 11 -- Common Mistakes

### Mistake 1 -- Treating Redis as only a message broker

Redis is primarily an in-memory data store and provides several data and messaging features.

It can be used as:

```
javascript.snippet JAVASCRIPT

CacheSession storeCelery brokerCelery result backendPub/Sub systemStreamsChannels layer

```

depending on the architecture.

---

### Mistake 2 -- Installing packages globally

Avoid:

```
javascript.snippet JAVASCRIPT

pip3 install django

```

for every project globally.

Prefer:

```
javascript.snippet JAVASCRIPT

python3 -m venv .venvs
source .venv/bin/activate
python -m pip install django

```

---

### Mistake 3 -- Forgetting to start Redis

---

Before testing:

```
javascript.snippet JAVASCRIPT
redis-cli ping
```

Make sure you get:

```
javascript.snippet JAVASCRIPT
PONG
```

---

## Mistake 4 -- Forgetting the Celery worker

Configuring Celery is not enough.

You also need a worker:

```
javascript.snippet JAVASCRIPT
celery -A myproject worker -l info
```

---

## Mistake 5 -- Confusing Cache and Celery

Django cache:

```
javascript.snippet JAVASCRIPT
Django Redis Cached data
```

Celery:

```
javascript.snippet JAVASCRIPT
Django Redis Broker Celery Worker
```

They can use the same Redis server but perform different jobs.

---

## Part 12 -- Quick Revision

---

# Message Broker

● ● ● javascript.snippet

JAVASCRIPT

Producer      Message Broker      Consumer

Purpose:

● ● ● javascript.snippet

JAVASCRIPT

DecouplingAsynchronous processingMessage deliveryScalabilityReliability features

---

## RabbitMQ

● ● ● javascript.snippet

JAVASCRIPT

General-purpose message broker

---

## Redis

● ● ● javascript.snippet

JAVASCRIPT

In-memory data store+Messaging features+Can be used [by](#) Celery+Can be used [for](#) Django  
cache  
+Can be used [by](#) Channels

---

## Kafka

● ● ● javascript.snippet

JAVASCRIPT

Distributed event streaming platform

---

## Celery

---

```
javascript.snippet JAVASCRIPT

Distributed task queue
```

Typical architecture:

```
javascript.snippet JAVASCRIPT

Django Celery Redis/RabbitMQ Celery Worker
```

---

## Django Cache

```
javascript.snippet JAVASCRIPT

from django.core.cache import cache
cache.set("key", "value", timeout=60)
value = cache.get("key")
```

---

## Django Session with Cache

```
javascript.snippet JAVASCRIPT

SESSION_ENGINE = ("django.contrib.sessions.backends.cache")
SESSION_CACHE_ALIAS = "default"
```

---

## Celery Broker

```
javascript.snippet JAVASCRIPT

CELERY_BROKER_URL = ("redis://127.0.0.1:6379/0")
```

---

## Django Channels

---

```
javascript.snippet JAVASCRIPT

CHANNEL_LAYERS = {    "default": {    "BACKEND": (
"channels_redis.core.RedisChannelLayer"
    ),    "CONFIG": {    "hosts": [    ("127.0.0.1", 6379)

],    },    },}
```

## Final Mental Model

```
javascript.snippet JAVASCRIPT

APPLICATION

Message / Task

Redis

Cache Celery Channels

Worker WebSocket
```

## One-line definitions

```
javascript.snippet JAVASCRIPT

Message Broker Middleware that transports messages between producers and consumers.
Producer
Sends/creates messages.Consumer Receives/processes messages.Redis In-memory data store
with caching and messaging capabilities.Celery Distributed task queue for background/
asynchronous
jobs.RabbitMQ General-purpose message broker.Kafka Distributed event streaming
platform
.Django Cache Stores temporary/reusable data to reduce expensive operations.Django
Channels
Adds support for protocols/use cases such as WebSockets and real-time communication.
Redis
Channel Layer Allows Channels instances/consumers to communicate through Redis.
```